

Formalización de la confluencia $\beta\eta$ en Lean

Maximiliano Onofre Martínez

Tabla de contenidos

Introducción	1
1 Relaciones	2
1.1. Definiciones básicas	2
2 Cálculo λ	6
2.1. La representación <i>locally nameless</i>	6
3 Confluencia $\beta\eta$	11
3.1. Confluencia fuerte de η	11
3.2. Conmutación fuerte de η y β	21

Introducción

1 Relaciones

1.1. Definiciones básicas

Definición 1.1.1. Una relación binaria sobre un tipo α es una función $\alpha \rightarrow \alpha \rightarrow \text{Prop}$, es decir, una proposición sobre pares de elementos de α .

```
variable { $\alpha$  : Type} {r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ }
```

Definición 1.1.2. Sea r una relación en α . Entonces r induce las siguientes relaciones binarias.

I. $\text{ReflGen } r$ es la cerradura reflexiva de r , definida inductivamente de la siguiente manera:

```
inductive ReflGen { $\alpha$  : Type} (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$   
  | refl {a :  $\alpha$ } : ReflGen r a a  
  | single {a b :  $\alpha$ } : r a b  $\rightarrow$  ReflGen r a b
```

II. $\text{ReflTransGen } r$ es la cerradura reflexiva y transitiva de r , definida inductivamente de la siguiente manera:

```
inductive ReflTransGen { $\alpha$  : Type} (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$   
  | refl {a :  $\alpha$ } : ReflTransGen r a a  
  | tail {a b c :  $\alpha$ } : ReflTransGen r a b  $\rightarrow$  r b c  $\rightarrow$  ReflTransGen r a c
```

Teorema 1.1.1. Si dos términos están relacionados por la cerradura reflexiva de r , entonces también lo están por su cerradura reflexiva y transitiva.

```
theorem ReflGen.to_reflTransGen :  
   $\forall \{a b : \alpha\}, \text{ReflGen } r a b \rightarrow \text{ReflTransGen } r a b$ 
```

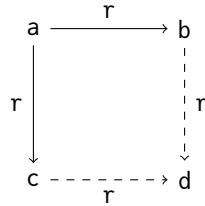
Las relaciones anteriores pueden describirse mediante caminos de a a b :

- $r a b$ indica la existencia de un camino de longitud 1.
- $\text{ReflGen } r a b$, de un camino de longitud 0 o 1.
- $\text{ReflTransGen } r a b$, de un camino de longitud n , para algún $n \in \mathbb{N}$.

Representamos estas relaciones mediante $a \rightarrow b$, $a \rightarrow^= b$ y $a \rightarrow\!\!\gg b$, respectivamente.

Definición 1.1.3. Sea r una relación en α .

Decimos que r tiene la *propiedad del diamante* si, para cualesquiera a, b y c tales que $a \rightarrow b$ y $a \rightarrow c$, existe un elemento d tal que $b \rightarrow d$ y $c \rightarrow d$. Visualmente, esta propiedad afirma que el siguiente diagrama siempre puede completarse:

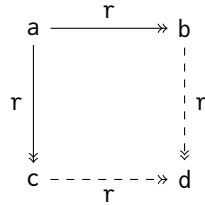


```
def Diamond (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) :=
   $\forall \{a\ b\ c : \alpha\},\ r\ a\ b \rightarrow r\ a\ c \rightarrow \exists\ d,\ r\ b\ d \wedge r\ c\ d$ 
```

Confluencia y Conmutación

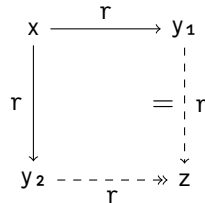
Definición 1.1.4. Sea r una relación en α .

I. r es **confluente** si su cerradura reflexiva y transitiva tiene la propiedad del diamante.



```
def Confluent (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) := Diamond (ReflTransGen r)
```

II. r es **fuertemente confluente** si:



1.1. Definiciones básicas

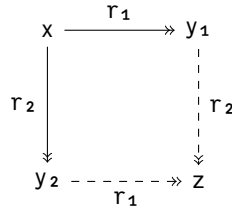
```
def StronglyConfluent (r :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) :=
   $\forall \{x \ y_1 \ y_2\}, r \ x \ y_1 \rightarrow r \ x \ y_2 \rightarrow \exists z, \text{ReflGen } r \ y_1 \ z \wedge \text{ReflTransGen } r \ y_2 \ z$ 
```

Teorema 1.1.2. Si una relación es fuertemente confluente, entonces también es confluente.

```
theorem StronglyConfluent.toConfluent (h : StronglyConfluent r) : Confluent r
```

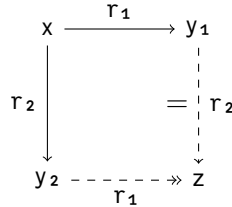
Definición 1.1.5. Sean r_1 y r_2 dos relaciones en α .

I. r_1 y r_2 **conmutan** si:



```
def Commute (r1 r2 :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) :=
   $\forall \{x \ y_1 \ y_2\}, \text{ReflTransGen } r_1 \ x \ y_1 \rightarrow \text{ReflTransGen } r_2 \ x \ y_2 \rightarrow$ 
   $\exists z, \text{ReflTransGen } r_2 \ y_1 \ z \wedge \text{ReflTransGen } r_1 \ y_2 \ z$ 
```

II. r_1 y r_2 **conmutan fuertemente** si:



```
def StronglyCommute (r1 r2 :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ ) :=
   $\forall \{x \ y_1 \ y_2\}, r_1 \ x \ y_1 \rightarrow r_2 \ x \ y_2 \rightarrow \exists z, \text{ReflGen } r_2 \ y_1 \ z \wedge \text{ReflTransGen } r_1 \ y_2 \ z$ 
```

Teorema 1.1.3. Si dos relaciones conmutan fuertemente, entonces también conmutan.

```
theorem StronglyCommute.toCommute {r1 r2 :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ }
  (h : StronglyCommute r1 r2) : Commute r1 r2
```

Teorema 1.1.4. La conmutación entre relaciones es simétrica, es decir, si r_1 y r_2 conmutan, entonces r_2 y r_1 también.

```
theorem Commute.symmetric : Symmetric (@Commute  $\alpha$ )
```

Teorema 1.1.5 (Hindley-Rosen). Sean r_1 y r_2 dos relaciones sobre un mismo tipo. Si ambas son confluentes y conmutan entre sí, entonces su unión $r_1 \sqcup r_2$ es confluente.

```
theorem Commute.join_confluent { $r_1$   $r_2$  :  $\alpha \rightarrow \alpha \rightarrow \text{Prop}$ }  
  ( $c_1$  : Confluent  $r_1$ ) ( $c_2$  : Confluent  $r_2$ ) (comm : Commute  $r_1$   $r_2$ ) :  
    Confluent ( $r_1 \sqcup r_2$ )
```

2 Cálculo λ

2.1. La representación *locally nameless*

Definición 2.1.1. Utilizamos una representación *locally nameless* para el cálculo λ , donde las variables ligadas se representan como números naturales (índices de De Bruijn) y las variables libres como términos del tipo `Var`. Sobre el tipo `Var` la igualdad es decidible (`DecidableEq`) y es posible generar una variable fuera de cualquier conjunto finito dado (`HasFresh`).

```
universe u
variable {Var : Type u} [HasFresh Var] [DecidableEq Var]

inductive Term (Var : Type u)
| bvar : ℕ → Term Var
| fvar : Var → Term Var
| abs  : Term Var → Term Var
| app  : Term Var → Term Var → Term Var
```

Por ejemplo, podemos representar la expresión “ $\lambda x. x y$ ” de la siguiente manera.

```
variable {y : Var}
def ejemplo_rep := abs (app (bvar 0) (fvar y))
```

Lo cual también se puede escribir como “ $\lambda. 0 y$ ”.

Definición 2.1.2. La sustitución consiste en remplazar una variable libre por un término.

```
def subst (m : Term Var) (x : Var) (sub : Term Var) : Term Var :=
  match m with
  | bvar i  => bvar i
  | fvar x' => if x = x' then sub else fvar x'
  | app l r => app (subst l x sub) (subst r x sub)
  | abs M   => abs (subst M x sub)
```

Definimos una notación para la sustitución de variables libres que imita la notación matemática estándar.

2.1. La representación *locally nameless*

```
scoped notation t:max "[" x " := " t' "]" => subst t x t'
```

Definición 2.1.3. La función `fv`, definida a continuación, calcula el conjunto de variables libres de una expresión. Dado que utilizamos una representación *locally nameless*, donde las variables ligadas se representan como índices, cualquier nombre que veamos es una variable libre de un término.

```
def fv : Term Var → Finset Var
| bvar _ => {}
| fvar x => {x}
| abs e1 => fv e1
| app l r => fv l ∪ fv r
```

El tipo `Finset Var` representa un conjunto finito de elementos del tipo `Var`.

Definición 2.1.4. La operación de apertura sustituye un índice por un término.

```
def openRec (i : ℕ) (sub : Term Var) : Term Var → Term Var
| bvar i' => if i = i' then sub else bvar i'
| fvar x  => fvar x
| app l r => app (openRec i sub l) (openRec i sub r)
| abs M   => abs (openRec (i+1) sub M)
```

Su notación es la siguiente.

```
scoped notation:68 e "[" i " ↪ " sub "]" => openRec i sub e
```

Las aplicaciones habituales de esta función sustituyen el índice cero por una variable o un término λ . La siguiente definición es una forma práctica de indicar estos usos.

```
def open' {X} (e u) := @openRec X 0 u e

scoped infixr:80 " ^ " => open'
```

Así, $(e \wedge x)$ puede leerse como “reemplaza el índice 0 por la variable x en e ” o “abre e con la variable x ”.

Definición 2.1.5. La operación de cierre reemplaza la variable libre x por el índice k . Es la operación inversa de la apertura.

```
def closeRec (k : ℕ) (x : Var) : Term Var → Term Var
| fvar x' => if x = x' then bvar k else fvar x'
| bvar i  => bvar i
| app l r => app (closeRec k x l) (closeRec k x r)
| abs t   => abs (closeRec (k+1) x t)
```

Cuya notación es la siguiente.

```
scoped notation:68 e "[[ k " ↵ " x " ]]" => closeRec k x e
```

Al igual que con la apertura, el uso habitual es sobre el índice cero. La siguiente definición trata ese caso.

```
def close (e u) := @closeRec Var _ 0 u e

scoped infixr:80 " ^* " => close
```

Teorema 2.1.1. Abrir un término con una variable que no aparece libre en él y después cerrar sobre esa misma variable devuelve el término original.

```
theorem open_close_var (x : Var) (M : Term Var) (h : x ∉ fv M) :
  M = (M ^ fvar x) ^* x
```

Definición 2.1.6 (Local closure). Un término se denomina “localmente cerrado” cuando no contiene índices no ligados. La proposición LC e se cumple cuando una expresión e es localmente cerrada.

```
inductive LC : Term Var → Prop
| fvar (x : Var) : LC (fvar x)
| abs (L : Finset Var) (e : Term Var) : (∀ x ∉ L, LC (e ^ fvar x)) → LC (abs e)
| app {l r} : l.LC → r.LC → LC (app l r)
```

Definición 2.1.7. Dada una relación r en el tipo $Term Var$, se llama cerradura compatible de r a la relación $CompatGen\ r$, definida de la siguiente manera.

```
inductive CompatGen (r : Term Var → Term Var → Prop) : Term Var → Term Var → Prop
| base : r M N → CompatGen r M N
| appL : LC Z → CompatGen r M N → CompatGen r (app M Z) (app N Z)
| appR : LC Z → CompatGen r M N → CompatGen r (app Z M) (app Z N)
| xi (L : Finset Var) : (∀ x ∉ L, CompatGen r (M ^ fvar x) (N ^ fvar x)) →
  CompatGen r (abs M) (abs N)
```

2.1. La representación *locally nameless*

Definición 2.1.8. Sea Beta una relación en Term Var, definida de la siguiente manera.

```
inductive Beta : Term Var → Term Var → Prop
| beta : LC (abs A) → LC B → Beta (app (abs A) B) (A ^ B)
```

I. La reducción β de un solo paso es la cerradura compatible de Beta.

```
@[reduction_sys "β"]
abbrev FullBeta : Term Var → Term Var → Prop := CompatGen Beta
```

II. La reducción β es la cerradura reflexiva y transitiva de CompatGen Beta, es decir, ReflTransGen FullBeta.

Definición 2.1.9. Sea Eta una relación en Term Var, definida de la siguiente manera.

```
inductive Eta : Term Var → Term Var → Prop
| eta : LC M → Eta (abs (app M (bvar 0))) M
```

I. La reducción η de un solo paso es la cerradura compatible de Eta.

```
@[reduction_sys "η"]
abbrev FullEta : Term Var → Term Var → Prop := CompatGen Eta
```

II. La reducción η es la cerradura reflexiva y transitiva de CompatGen Eta, esto es ReflTransGen FullEta.

Teorema 2.1.2. El lado derecho de una reducción η es localmente cerrado.

```
theorem step_lc_r {M M' : Term Var} (step : M →η M') : LC M'
```

Teorema 2.1.3. Una reducción de un solo paso desde $\text{abs } (\text{app } N \text{ (bvar } 0))$ produce N, o bien $\text{abs } (\text{app } N' \text{ (bvar } 0))$ para alguna N' tal que $N \rightarrow_{\eta} N'$.

```
theorem invert_step_eta_redex {M N : Term Var} (lc_N : LC N)
  (step : abs (app N (bvar 0)) →η M) :
  M = N ∨ ∃ N', N →η N' ∧ M = abs (app N' (bvar 0))
```

Teorema 2.1.4. Si $M \rightarrow_{\eta} M'$, entonces al cerrar ambos términos sobre una variable y abstraer se obtiene la reducción $\text{abs } (M \text{ }^* x) \rightarrow_{\eta} \text{abs } (M' \text{ }^* x)$.

```
theorem step_abs_close {M M' : Term Var} {x : Var} (step : M  $\rightarrow^f$  M') (lc_M : LC M) :
  abs (M  $\wedge^*$  x)  $\rightarrow^f$  abs (M'  $\wedge^*$  x)
```

Elección de variables nuevas

Llamamos *nueva* a una variable que todavía no se usa en la demostración. Al pedir en la definición 2.1.1 que pueda generarse una variable fuera de cualquier conjunto finito, nos aseguramos de que siempre haya una variable nueva disponible cuando se necesite. El teorema `fresh_exists` enuncia esta propiedad:

```
theorem fresh_exists (L : Finset Var) :  $\exists$  x, x  $\notin$  L
```

El conjunto al que lo aplicamos lo construye `free_union [fv] Var`: la unión de todas las variables, todos los conjuntos finitos y las variables libres de todos los términos presentes en la demostración.

Por ejemplo, supongamos que en una demostración figuran una variable x , un conjunto finito L y un término M . La siguiente línea aplica el teorema anterior:

```
have <w, hw> := fresh_exists (free_union [fv] Var)
```

Esto produce una variable w junto con la hipótesis:

```
hw : w  $\notin$   $\emptyset \cup \{x\} \cup L \cup \text{fv } M$ 
```

En adelante, cuando escribamos “sea w una variable nueva”, nos referimos a esta construcción. Las condiciones particulares que se necesiten después ($w \notin L$, $w \notin \text{fv } M$, etcétera) son consecuencias inmediatas de `hw`, y las resuelve la táctica `aesop` (*Automated Extensible Search for Obvious Proofs*), un procedimiento que busca demostraciones de manera automática.

3 Confluencia $\beta\eta$

3.1. Confluencia fuerte de η

Teorema 3.1.1. La reducción η de un solo paso es fuertemente confluente.

Demostración. Supongamos que M , M_1 y M_2 son términos lambda tales que $M \rightarrow_{\eta^f} M_1$ y $M \rightarrow_{\eta^f} M_2$. Entonces existe un término lambda z tal que $\text{ReflGen FullEta } M_1 \ z \text{ y } M_2 \rightarrow_{\eta^f} z$.

Código Lean

```
theorem strong_confluence_eta :
  StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
```

Estado de la prueba

```
M M1 M2 : Term Var
h1 : M →ηf M1
h2 : M →ηf M2
⊢ ∃ z, ReflGen FullEta M1 z
  ∧ M2 →ηf z
```

Por el teorema 1.1.1 ($\text{ReflGen.to_reflTransGen}$), basta encontrar un término z tal que $\text{ReflGen FullEta } M_1 \ z$ y $\text{ReflGen FullEta } M_2 \ z$.

Código Lean

```
theorem strong_confluence_eta :
  StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  suffices h : ∃ z,
    ReflGen FullEta M1 z ∧ ReflGen FullEta M2 z by
  have ⟨n, hn1, hn2⟩ := h
  use n -- ⊢
  constructor
  · exact hn1
  · apply ReflGen.to_reflTransGen
    exact hn2
  induction h1 generalizing M2
```

Estado de la prueba

```
M M1 M2 n : Term Var
h1 : M →ηf M1
h2 : M →ηf M2
h : ∃ z,
  ReflGen FullEta M1 z
  ∧ ReflGen FullEta M2 z
hn1 : ReflGen FullEta M1 n
hn2 : ReflGen FullEta M2 n
⊢ ReflGen FullEta M1 n
  ∧ M2 →ηf n
```

Encontraremos dicho término por inducción sobre la derivación de $h_1 : M \rightarrow_{\eta^f} M_1$.

Demostraremos que, para todo término M_2 , si $M \rightarrow_{\eta^f} M_2$, entonces existe un término z tal que $\text{ReflGen FullEta } M_1 \ z$ y $\text{ReflGen FullEta } M_2 \ z$. Procedemos por casos según la última regla empleada en la derivación de h_1 .

3.1. Confluencia fuerte de η

I. Si la última regla fue base, entonces $M = \text{abs } (\text{app } P \text{ (bvar } 0))$ y $M_1 = P$, para alguna P tal que $LC \ P$.

```
theorem strong_confluence_eta :
  StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  :
  case base _ _ P' P e_P =>
    have ⟨lc_P⟩ := e_P
    have h := invert_step_eta_redex lc_P h2
```

```
M M1 M2 P' P : Term Var
lc_P : LC P
e_P : Eta P' P
h2 : abs (app P (bvar 0)) →η M2
h : M2 = P ∨ ∃ N', P →η N' ∧ M2 = abs (app N' (bvar 0))
⊢ ∃ z, ReflGen FullEta P z ∧ ReflGen FullEta M2 z
```

Aplicando el teorema 2.1.3 (`invert_step_eta_redex`) a h_2 obtenemos dos posibilidades: $M_2 = P$, o bien $M_2 = \text{abs } (\text{app } P_1 \text{ (bvar } 0))$ para alguna P_1 con $P \rightarrow^{\eta} P_1$. Analizamos cada una.

(a) Si $M_2 = P$, ambas derivaciones aplicaron la regla η a $\text{abs } (\text{app } P \text{ (bvar } 0))$ y llegaron al mismo término. Basta tomar $z = P$.

Código Lean

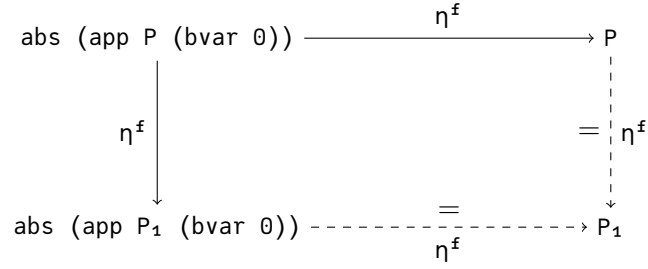
```
case base _ _ P' P e_P =>
  have ⟨lc_P⟩ := e_P
  have h := invert_step_eta_redex lc_P h2
  cases h with
  | inl h_eq =>
    rw [h_eq] -- ⊢
    use P
```

Estado de la prueba

```
M M1 M2 P' P : Term Var
lc_P : LC P
e_P : Eta P' P
h2 : abs (app P (bvar 0))
      →η M2
h_eq : M2 = P
⊢ ∃ z, ReflGen FullEta P z
  ∧ ReflGen FullEta P z
```

(b) Si $M_2 = \text{abs } (\text{app } P_1 \text{ (bvar } 0))$ para alguna P_1 con $P \rightarrow^{\eta} P_1$, tomamos $z = P_1$: el primer componente de la conjunción es $P \rightarrow^{\eta} P_1$, y el segundo se obtiene aplicando la regla η a $\text{abs } (\text{app } P_1 \text{ (bvar } 0))$, cuya premisa $LC \ P_1$ se extrae del teorema 2.1.2 (`step_lc_r`) aplicado a $r_P P_1$.

3.1. Confluencia fuerte de η



```

cases h with
| inl h_eq =>
  rw [h_eq]
  use P
| inr h =>
  have ⟨P1, r_P_P1, h_eq⟩ := h
  rw [h_eq]
  use P1 -- ⊢
  constructor
  · exact .single r_P_P1
  · exact .single (.base (.eta (step_lc_r r_P_P1)))

```

```

M M1 M2 P' P P1 : Term Var
lc_P : LC P
e_P : Eta P' P
h2 : abs (app P (bvar 0)) →ηf M2
h : ∃ N', P →ηf N' ∧ M2 = abs (app N' (bvar 0))
r_P_P1 : P →ηf P1
h_eq : M2 = abs (app P1 (bvar 0))
⊢ ReflGen FullEta P P1 ∧ ReflGen FullEta (abs (app P1 (bvar 0))) P1

```

II. Si la última regla fue appL, entonces $M = \text{app } P \ N$ y $M_1 = \text{app } P_1 \ N$, para alguna P , P_1 y N tales que $LC \ N$ y $P \rightarrow^{\eta^f} P_1$.

```

theorem strong_confluence_eta : StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  :
  case appL N P P1 lc_N r_P_P1 ih =>

```

```

M M1 M2 N P P1 : Term Var
lc_N : LC N
r_P_P1 : P →ηf P1
ih : ∀ {M2 : Term Var}, P →ηf M2 → ∃ z,

```

3.1. Confluencia fuerte de η

```

ReflGen FullEta P1 z ∧ ReflGen FullEta M2 z
h2 : app P N →ηf M2
⊢ ∃ z, ReflGen FullEta (app P1 N) z ∧ ReflGen FullEta M2 z

```

A continuación, procedemos por casos sobre la derivación de $\text{app } P \ N \rightarrow^{\eta^f} M_2$:

(a) Si la última regla fue appL , entonces $M_2 = \text{app } P_2 \ N$ tal que $P \rightarrow^{\eta^f} P_2$ y $\text{LC } N$. Por la hipótesis de inducción aplicada sobre $P \rightarrow^{\eta^f} P_2$, sabemos que existe un término P_3 tal que $\text{ReflGen FullEta } P_1 \ P_3$ y $\text{ReflGen FullEta } P_2 \ P_3$. Por lo tanto, tomamos $z = \text{app } P_3 \ N$.

```

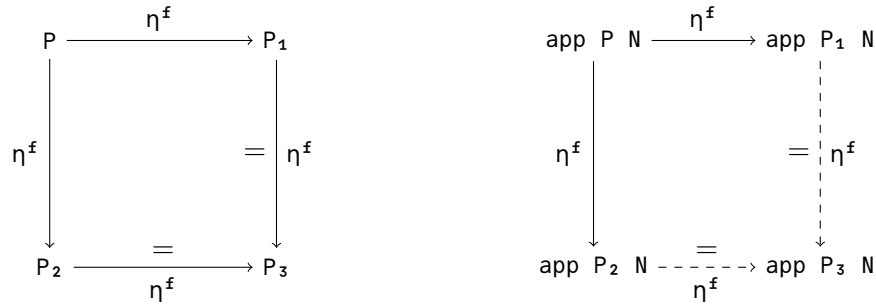
case appL N P P1 lc_N r_P_P1 ih =>
  case appL P2 r_P_P2 _ =>
    have ⟨P3, refl_P1_P3, refl_P2_P3⟩ := ih r_P_P2
    use app P3 N

```

```

M M1 N P P1 P2 P3 : Term Var
lc_N : LC N
r_P_P1 : P →ηf P1
ih : ∀ {M2 : Term Var}, P →ηf M2 → ∃ z,
  ReflGen FullEta P1 z ∧ ReflGen FullEta M2 z
r_P_P2 : P →ηf P2
refl_P1_P3 : ReflGen FullEta P1 P3
refl_P2_P3 : ReflGen FullEta P2 P3
⊢ ReflGen FullEta (app P1 N) (app P3 N) ∧ ReflGen FullEta (app P2 N) (app P3 N)

```



Para demostrar cada componente de la conjunción procedemos por casos sobre $\text{ReflGen FullEta } P_1 \ P_3$ y $\text{ReflGen FullEta } P_2 \ P_3$, respectivamente: si son reflexivas, ambos lados ya coinciden; si constan de un paso, reducimos P_1 a P_3 en la primera y P_2 a P_3 en la segunda, dejando fija N , mediante la regla appL .

3.1. Confluencia fuerte de η

```

case appL N P P1 lc_N r_P_P1 ih =>
  case appL P2 r_P_P2 _ =>
    have ⟨P3, refl_P1_P3, refl_P2_P3⟩ := ih r_P_P2
    use app P3 N
    constructor
    • cases refl_P1_P3
      case refl => rfl
      case single r_P1_P3 => exact .single (.appL lc_N r_P1_P3)
    • cases refl_P2_P3
      case refl => rfl
      case single r_P2_P3 => exact .single (.appL lc_N r_P2_P3)

```

(b) Si la última regla fue appR, entonces $M_2 = \text{app } P \ N_1$, para alguna N_1 donde $LC \ P$ y $N \rightarrow_{\eta^f} N_1$. Así, tomamos $z = \text{app } P_1 \ N_1$.

```

case appL N P P1 lc_N r_P_P1 ih =>
  case appL P2 r_P_P2 _ =>
  :
  case appR N1 _ r_N_N1 =>

```

```

M M1 N N1 P P1 : Term Var
lc_N : LC N
r_P_P1 : P →ηf P1
ih : ∀ {M2 : Term Var}, P →ηf M2 → ∃ z,
  ReflGen FullEta P1 z ∧ ReflGen FullEta M2 z
lc_P : LC P
r_N_N1 : N →ηf N1
⊢ ReflGen FullEta (app P1 N) (app P1 N1) ∧ ReflGen FullEta (app P N1) (app P P1 N1)

```

$$\begin{array}{ccc}
 \text{app } P \ N & \xrightarrow{\eta^f} & \text{app } P_1 \ N \\
 \downarrow \eta^f & & \downarrow \eta^f \\
 \text{app } P \ N_1 & \xrightarrow[\eta^f]{=} & \text{app } P_1 \ N_1
 \end{array}$$

Cada componente de la conjunción se obtiene con una sola reducción: en la primera reducimos N a N_1 dejando fija P_1 , y en la segunda reducimos P a P_1 dejando fija N_1 .

3.1. Confluencia fuerte de η

Las premisas de cerradura local que piden las reglas `appR` y `appL` se obtienen del teorema 2.1.2 (`step_lc_r`): de $r_P_P_1$ obtenemos $LC\ P_1$ y de $r_N_N_1$ obtenemos $LC\ N_1$.

(c) Si la última regla fue `base`, la regla η se habría aplicado directamente a `app P N`, lo cual es imposible: la regla η solo reduce abstracciones y este término es una aplicación. Así, no hay nada que demostrar.

```
case appL N P P1 lc_N r_P_P1 ih =>
  case appL P2 r_P_P2 _ =>
    :
  case appR N1 _ r_N_N1 =>
    use app P1 N1
    constructor
    • exact .single (.appR (step_lc_r r_P_P1) r_N_N1)
    • exact .single (.appL (step_lc_r r_N_N1) r_P_P1)
  case base => contradiction
```

III. Si la última regla fue `appR`, entonces $M = \text{app } N\ P$ y $M_1 = \text{app } N\ P_1$, para alguna N , P y P_1 tales que $LC\ N$ y $P \rightarrow_{\eta^f} P_1$.

```
theorem strong_confluence_eta : StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  :
  case appR N P P1 lc_N r_P_P1 ih =>
```

```
M M1 M2 N P P1 : Term Var
lc_N : LC N
r_P_P1 : P  $\rightarrow_{\eta^f}$  P1
ih :  $\forall \{M_2 : \text{Term Var}\}, P \rightarrow_{\eta^f} M_2 \rightarrow \exists z,$ 
      ReflGen FullEta P1 z  $\wedge$  ReflGen FullEta M2 z
h2 : app N P  $\rightarrow_{\eta^f}$  M2
 $\vdash \exists z, \text{ReflGen FullEta (app N P}_1) z \wedge \text{ReflGen FullEta M}_2 z$ 
```

Este caso es análogo al anterior: basta intercambiar el argumento que se reduce con el que queda fijo y, con ellos, las reglas `appL` y `appR`. La formalización es la siguiente.

```
theorem strong_confluence_eta : StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  :
  case appR N' P P1 lc_N' r_P_P1 ih =>
    cases h2
    case appL N1 r_N_N1 _ =>
      use app N1 P1
```

```

    constructor
    · exact .single (.appL (step_lc_r r_P_P1) r_N_N1)
    · exact .single (.appR (step_lc_r r_N_N1) r_P_P1)
case appR P2 r_P_P2 =>
  have ⟨P3, refl_P1_P3, refl_P2_P3⟩ := ih r_P_P2
  use app N' P3
  constructor
  · cases refl_P1_P3
    case refl => rfl
    case single r_P1_P3 => exact .single (.appR lc_N' r_P1_P3)
  · cases refl_P2_P3
    case refl => rfl
    case single r_P2_P3 => exact .single (.appR lc_N' r_P2_P3)
case base => contradiction

```

IV. Si la última regla fue xi, entonces $M = \text{abs } P$ y $M_1 = \text{abs } Q$, para alguna P y Q tales que, para toda variable x fuera de un conjunto finito L , se cumple $(P \wedge \text{fvar } x) \rightarrow \eta^f (Q \wedge \text{fvar } x)$.

```

theorem strong_confluence_eta :
  StronglyConfluent (@FullEta Var) := by
  intro M M1 M2 h1 h2
  :
  case xi P Q L h_xi ih =>

```

```

M M1 M2 P Q : Term Var
L : Finset Var
h_xi :  $\forall x \notin L, (P \wedge \text{fvar } x) \rightarrow \eta^f (Q \wedge \text{fvar } x)$ 
h2 :  $\text{abs } P \rightarrow \eta^f M_2$ 
 $\vdash \exists z, \text{ReflGen FullEta } (\text{abs } Q) z \wedge \text{ReflGen FullEta } M_2 z$ 

```

Procedemos por casos sobre la derivación de $\text{abs } P \rightarrow \eta^f M_2$ y, como se trata de una abstracción, solo pueden aplicarse las reglas base o xi.

(a) Si la última regla fue base, entonces M_2 se obtuvo aplicando la regla η a $\text{abs } P$, lo cual exige que $P = \text{app } M_2 \text{ (bvar } 0)$ y $\text{LC } M_2$.

```

case xi P Q L h_xi ih =>
  cases h2
  case base e_P =>
    have h1 := CompatGen.xi L h_xi
    have ⟨lc_M2⟩ := e_P
    have h := invert_step_eta_redex lc_M2 h1

```

3.1. Confluencia fuerte de η

```

M M1 M2 P Q : Term Var
L : Finset Var
e_P : Eta (abs P) M2
lc_M2 : LC M2
h_xi : ∀ x ∉ L, (app M2 (bvar 0) ^ fvar x) →η (Q ^ fvar x)
h1 : abs (app M2 (bvar 0)) →η abs Q
h : abs Q = M2 ∨ ∃ N', M2 →η N' ∧ abs Q = abs (app N' (bvar 0))
⊢ ∃ z, ReflGen FullEta (abs Q) z ∧ ReflGen FullEta M2 z

```

Estamos ante el mismo escenario del caso I-(b), ahora h_1 parte de $\text{abs } (\text{app } M_2 \text{ (bvar 0)})$ y el teorema 2.1.3 (`invert_step_eta_redex`) nos da dos posibilidades: $\text{abs } Q = M_2$, o bien $\text{abs } Q = \text{abs } (\text{app } R \text{ (bvar 0)})$ para alguna R con $M_2 \rightarrow^\eta R$. Analizamos cada una.

- Si $\text{abs } Q = M_2$, ambas reducciones llegaron al mismo término y basta tomar $z = M_2$.
- Si $\text{abs } Q = \text{abs } (\text{app } R \text{ (bvar 0)})$ con $M_2 \rightarrow^\eta R$, tomamos $z = R$: el primer componente de la conjunción se obtiene aplicando la regla η a $\text{abs } (\text{app } R \text{ (bvar 0)})$, cuya premisa $\text{LC } R$ se extrae del teorema 2.1.2 (`step_lc_r`) aplicado a $r_M_2_R$, y el segundo es la reducción $M_2 \rightarrow^\eta R$.

```

case base e_P =>
  :
  cases h with
  | inl h_eq =>
    rw [h_eq]
    use M2
  | inr h =>
    have ⟨R, r_M2_R, h_eq⟩ := h
    rw [h_eq]
    use R -- ⊢
    constructor
    · exact .single (.base (.eta (step_lc_r r_M2_R)))
    · exact .single r_M2_R

```

```

M M1 M2 P Q R : Term Var
L : Finset Var
e_P : Eta (abs P) M2
lc_M2 : LC M2
h_xi : ∀ x ∉ L, (app M2 (bvar 0) ^ fvar x) →η (Q ^ fvar x)
h1 : abs (app M2 (bvar 0)) →η abs Q
h : abs Q = M2 ∨ ∃ N', M2 →η N' ∧ abs Q = abs (app N' (bvar 0))
r_M2_R : M2 →η R

```

3.1. Confluencia fuerte de η

```

h_eq : abs Q = abs (app R (bvar 0))
⊢ ReflGen FullEta (abs (app R (bvar 0))) R ∧ ReflGen FullEta M2 R

```

(b) Si la última regla fue xi, entonces $M_2 = \text{abs } P_1$, para alguna P_1 tal que, para toda variable x fuera de un conjunto finito L' , se cumple $(P \wedge \text{fvar } x) \rightarrow_{\eta^f} (P_1 \wedge \text{fvar } x)$.

```

case xi P Q L h_xi ih =>
  cases h2
  :
  case xi P1 L' h_xi' =>

```

```

P P1 Q : Term Var
L L' : Finset Var
ih : ∀ x ∉ L, ∀ {M2 : Term Var}, (P ∧ fvar x) →ηf M2 →
  ∃ z, ReflGen FullEta (Q ∧ fvar x) z ∧ ReflGen FullEta M2 z
h_xi : ∀ x ∉ L, (P ∧ fvar x) →ηf (Q ∧ fvar x)
h_xi' : ∀ x ∉ L', (P ∧ fvar x) →ηf (P1 ∧ fvar x)
⊢ ∃ z, ReflGen FullEta (abs Q) z ∧ ReflGen FullEta (abs P1) z

```

Sea w una variable nueva. De h_xi' aplicada a w se sigue $(P \wedge \text{fvar } w) \rightarrow_{\eta^f} (P_1 \wedge \text{fvar } w)$ y, por la hipótesis de inducción, existe un término P_2 tal que $\text{ReflGen FullEta } (Q \wedge \text{fvar } w) P_2$ y $\text{ReflGen FullEta } (P_1 \wedge \text{fvar } w) P_2$. Por lo tanto, tomamos $z = \text{abs } (P_2 \wedge w)$.

```

case xi P1 L' h_xi' =>
  have ⟨w, hw⟩ := fresh_exists (free_union [fv] Var)
  have r_P_P1 := h_xi' w (by aesop)
  have ⟨P2, refl_Q_P2, refl_P1_P2⟩ := ih w (by aesop) r_P_P1
  use abs (P2 ∧ w)

```

$$\begin{array}{ccc}
 P \wedge \text{fvar } w & \xrightarrow{\eta^f} & Q \wedge \text{fvar } w \\
 \downarrow \eta^f & & \downarrow = \eta^f \\
 P_1 \wedge \text{fvar } w & \xrightarrow[\eta^f]{=} & P_2
 \end{array}
 \qquad
 \begin{array}{ccc}
 \text{abs } P & \xrightarrow{\eta^f} & \text{abs } Q \\
 \downarrow \eta^f & & \downarrow \text{---} \eta^f \text{---} \\
 \text{abs } P_1 & \xrightarrow[\eta^f]{=} & \text{abs } (P_2 \wedge w)
 \end{array}$$

Para probar el primer componente de la conjunción procedemos por casos sobre $\text{ReflGen FullEta } (Q \wedge \text{fvar } w) P_2$:

3.1. Confluencia fuerte de η

- Si la reducción es reflexiva, entonces $P_2 = Q \wedge \text{fvar } w$, de modo que $z = \text{abs } ((Q \wedge \text{fvar } w) \wedge^* w)$. Por el teorema 2.1.1 (`open_close_var`) este término es $\text{abs } Q$ y ambos lados coinciden.
- Si la reducción consta de un paso $\text{st} : (Q \wedge \text{fvar } w) \rightarrow_{\eta^f} P_2$, reescribimos $\text{abs } Q$ como $\text{abs } ((Q \wedge \text{fvar } w) \wedge^* w)$ mediante el teorema 2.1.1 (`open_close_var`), así, basta construir la reducción $\text{abs } ((Q \wedge \text{fvar } w) \wedge^* w) \rightarrow_{\eta^f} \text{abs } (P_2 \wedge^* w)$.

De h_{xi} aplicada a w obtenemos $(P \wedge \text{fvar } w) \rightarrow_{\eta^f} (Q \wedge \text{fvar } w)$, que garantiza LC $(Q \wedge \text{fvar } w)$, con esta premisa, el teorema 2.1.4 (`step_abs_close`) aplicado a st produce la reducción buscada.

```
case xi P1 L' h_xi' =>
:
constructor
• cases refl_Q_P2
  case refl => rw [← open_close_var w Q (by aesop)]
  case single st =>
    rw [open_close_var w Q (by aesop)]
    have r_P_Q := h_xi w (by aesop) -- ⊢
    exact .single (step_abs_close st (step_lc_r r_P_Q))
```

```
M M1 P P1 P2 Q : Term Var
L L' : Finset Var
w : Var
h_xi : ∀ x ∉ L, (P ∧ fvar x) →ηf (Q ∧ fvar x)
hw : w ∉ ∅ ∪ fv M ∪ fv M1 ∪ fv P ∪ fv P1 ∪ fv Q ∪ L ∪ L'
st : (Q ∧ fvar w) →ηf P2
r_P_Q : (P ∧ fvar w) →ηf (Q ∧ fvar w)
⊢ ReflGen FullEta (abs ((Q ∧ fvar w) ∧* w)) (abs (P2 ∧* w))
```

Para probar el segundo componente de la conjunción procedemos por casos sobre `ReflGen FullEta (P1 ∧ fvar w) P2`, el cual es análogo al caso anterior, con P_1 en lugar de Q .

```
case xi P1 L' h_xi' =>
:
constructor
• cases refl_Q_P2
  case refl => rw [← open_close_var w Q (by aesop)]
  case single st =>
    rw [open_close_var w Q (by aesop)]
    have r_P_Q := h_xi w (by aesop)
    exact .single (step_abs_close st (step_lc_r r_P_Q))
```

3.2. Conmutación fuerte de η y β

```
• cases refl_P1_P2
  case refl => rw [← open_close_var w P1 (by aesop)]
  case single st =>
    rw [open_close_var w P1 (by aesop)]
    exact .single (step_abs_close st (step_lc_r r_P_P1))
```

En cada caso encontramos un término al que M_1 y M_2 se reducen en cero o un paso, lo que demuestra la confluencia fuerte de η . $\square$

3.2. Conmutación fuerte de η y β

Teorema 3.2.1. Las reducciones η y β de un solo paso conmutan fuertemente.